

CS 463 Final Project - Development Journal

GitHub Repo: <https://github.com/JosephBec/CS-463-Final>

Deployed Site: <https://JosephBec.github.io/CS-463-Final/>

Session 1 - Initial setup and repo creation

Started by creating a new public GitHub repo for the final project. Set up GitHub Pages deployment from the main branch. Did a first commit just to make sure the repo was working and github pages would actually deploy. Nothing fancy, literally just an HTML file that has the word “test” in it.

I knew I wanted a single-page site since that seemed simpler than dealing with copying headers and footers across multiple HTML files. The project instructions say you can do either, and a one-pager with anchor links felt cleaner for a portfolio.

Again, the biggest goal here was just making sure that I could get a GitHub site deployed.

Session 2 - HTML structure and content

This is where I built out most of the actual page. I started with what I already knew from the homework assignments and worked from there.

What I built

The page has four sections, all on one page with a sticky navbar that scrolls to each one:

1. **About** - intro paragraph and a headshot photo, using Bootstrap's grid (row/col-md-4/col-md-8) to put the image on the left and the text on the right. The image approach came from homework 01 (the images exercise) and the grid layout is from the Bootstrap homework (06-bootstrap).
2. **Previous Work** - this is where I used the Bootstrap accordion component. We never used accordions in any homework. We did cards, modals, and grid in the Bootstrap assignments, but not the accordion. I had to look this up in the Bootstrap 5 docs. Each accordion item is a job I've held (adult foster care, construction, Amazon delivery, food service). I used data-bs-parent to make it so only one panel is open at a time.
3. **Projects** - three project cards in a row using Bootstrap's card component and the grid. The card structure came from homework 06 (01-cards.html), but I changed it up quite a bit. Removed the placeholder boxes, added card-footer with GitHub links, and used h-100 to make them all the same height. Each card has a real project - Melodarr, Ebook-to-Audiobook, and a messaging app from CS 314.
4. **Contact** - a form with name, email, and message fields. The form structure is loosely based on homework 01 (04-form.html) but I stripped out all the radio buttons and checkboxes since those don't make sense for a contact form. Added the required attribute to each input.

Navbar

The navbar uses navbar-expand-md and fixed-top. It collapses into a hamburger menu on smaller screens using the navbar-toggler button. We

used navbars in some of the in-class examples but the toggler specifically was not part of any homework. I had to figure out the `data-bs-toggle="collapse"` and `data-bs-target` attributes from the Bootstrap docs. The toggler needs the Bootstrap JS bundle to work, which I loaded at the bottom of the body.

I used `ms-auto` on the nav list to push the links to the right side. The navbar brand just says “Joseph Bec” and links back to the top.

Decisions

- Went with a single-page layout. Easier to manage and I didn't want to deal with jQuery includes or copying nav/footer HTML across files.
- Used semantic HTML throughout: header, main, footer, nav, section tags. This matters for accessibility, which we covered in homework 02.
- Each section has an id that matches the navbar anchor links so clicking “About” scrolls to `#about`, etc.
- Used h2 for section headings and h3 inside the accordion headers. Tried to keep the heading hierarchy correct.

Issues

- The fixed-top navbar was covering the top of the page content. Fixed it by adding `mt-5 pt-4` to the main container so there's enough top margin to clear the navbar.
- Initially put the HTML in the wrong file (committed and had to fix it in the next commit). I accidentally put in `main.js`.

Session 3 - CSS styling

Split the CSS into two files: `base.css` for global resets and variables, and `styles.css` for component-specific styles. We learned about CSS custom properties in homework 03, and I used that approach here.

base.css

Set up CSS the color scheme on :root . I went with a dark green (#2e5339) as the primary color and kept everything else neutral. The green shows up in the navbar, footer, accordion highlights, and the submit button. Also set box-sizing: border-box on everything and picked just picked a generic sans-serif font.

styles.css

Component-level styles:

- Navbar background overridden to the green color with !important.
- About section photo set to width: 100%; height: auto so it scales with the column on any screen size.
- Accordion active state uses var(--light-green) for the background so it's visually connected to the color scheme.
- Project cards get a light gray border.
- Contact form inputs are styled with full width (up to 500px), padding, and a gray border. The submit button matches the green theme and changes to gray on hover.
- Error message styles for the form validation (red text, smaller font size).
- Confirmation message styled in green and bold.
- Footer gets the green background with white text.

Decisions

- Two CSS files instead of one. Keeps the global reset/variables separate from layout-specific stuff. Easier to maintain.
- No media queries beyond what Bootstrap handles. Bootstrap's grid and navbar-expand-md already handle the responsive breakpoints. I was thinking about writing custom breakpoints but Bootstrap was already doing what I needed.
- Used !important on the navbar background color. I know that's generally not great practice, but Bootstrap's utility classes were annoyingly aggressive and this was the cleanest way to get around

it. I ended up removing the `bg-dark` class from the HTML later which would have made the `!important` unnecessary, but I left it since it was already working.

Session 4 - JavaScript form validation

Wrote `js/main.js` from scratch. This is the JavaScript interactivity requirement for the project. The form validation goes beyond what we did in homework 07 (the DOM exercises) where we just toggled classes and fetched data. Here I'm creating and removing DOM elements dynamically for error messages, which we did touch on a bit in 07 but the validation logic is new.

How it works

1. Grabs the form and all three inputs with `document.querySelector`.
2. Listens for the submit event and calls `handleSubmit`, which calls `event.preventDefault()` so the page doesn't reload.
3. Clears any previous error messages by selecting all `.error-message` spans and removing them.
4. Validates each field:
 - Name: checks if it's empty after trimming whitespace.
 - Email: checks if empty, then checks against a regex pattern `(/^[^\s@]+@[^\s@]+\.[^\s@]+$/)` to make sure it looks like a real email.
 - Message: checks if empty.
5. If any field fails, a span element with class `error-message` gets created with `document.createElement` and inserted right after the input using `input.after()`.
6. If everything passes, a confirmation paragraph gets created and appended after the form. The form resets, and the confirmation auto-removes after 4 seconds with `setTimeout`.

Beyond class material

The email regex validation is something we didn't cover in any homework. We used the `type="email"` HTML attribute in homework 01, but writing a custom regex pattern to validate the format on the JS side was new for me. I do not like regex, it's annoying. So I looked up email regex patterns on google.

The `element.after()` method for inserting error messages is something I found on google. In homework 07 we used `appendChild` and `insertBefore`, but `.after()` is cleaner when you want to put something right after a specific element.

The `setTimeout` for auto-removing the confirmation message was straightforward but we hadn't used timers in any of the homework assignments.

Issues

- At first I was appending error messages without clearing old ones, so submitting the form multiple times would stack up duplicate error messages. Fixed that by adding the `clearErrors` function that runs at the top of every submit.
- The email regex was tricky. I started with something too strict that was rejecting valid emails. Ended up going with a simple pattern that just checks for `something@something.something`. It's not bulletproof but it catches the obvious stuff like missing the `@` sign.

Session 5 - Accessibility fixes, responsive bug, and cleanup

This session was mostly about going through the page and catching things I missed.

Accessibility

I went through the HTML looking at what Wave would flag. Found a few problems:

- There was no h1 on the page. The sections all started at h2 which means there was a skipped heading level. Added an h1 with my name at the top of the main content.
- The project cards had h5 tags which skipped from h2 straight to h5. Changed those to h3 with Bootstrap's h5 class so they still look the same size but the heading hierarchy is correct.
- The closing tags on those headings were still `</h5>` after I changed the opening to `<h3>`. Mismatched tags. Fixed those.
- The GitHub links had `target="_blank"` but no `rel="noopener noreferrer"`, which is a security thing. Added that to all three.
- Removed the `bg-dark` class from the navbar since I was already overriding the background color in CSS anyway. One less thing fighting for control.

Responsive image bug

When I resized the browser to a narrow width, the headshot photo wasn't scaling down. It was staying at a fixed size and overlapping the text below it. The issue was `max-width: 250px` in the CSS. That capped the image at 250px but didn't make it shrink below that. Changed it to `width: 100%; height: auto` so it fills whatever column Bootstrap gives it and scales down properly on small screens.

README

Updated the README with the project description, deployed site URL, how to run it, technologies used, and outside sources. The instructions say the README needs all of that.

Project structure

CS-463-Final/

```
|— index.html
|— css/
|   |— base.css
|   |— styles.css
|— js/
|   |— main.js
|— assets/
|   |— Black shirt headshot.jpeg
```

One HTML file, two CSS files, one JS file. Bootstrap 5.0.2 loaded from CDN (both the CSS and the JS bundle). No jQuery since I didn't end up needing it for a single-page site.

Things that go beyond what we learned in class

- **Bootstrap accordion** - We did cards, grids, and modals in homework 06, but never the accordion component. I had to learn the accordion, accordion-item, accordion-header, accordion-button, accordion-collapse, and accordion-body class structure from the Bootstrap docs. The data-bs-parent attribute controls the only one open at a time behavior.
- **Navbar toggler** - The hamburger menu that appears on small screens. We used navbars in class examples but the collapsible toggler with data-bs-toggle="collapse" wasn't part of any homework.

- **JavaScript form validation with regex** - Homework 07 covered DOM manipulation and event listeners, but custom form validation with regex email checking, dynamic error messages, and auto-dismissing confirmations was all new.
 - **CSS custom properties for theming** - We learned about custom properties in homework 03, but I used them here as an actual color theme system across multiple files, which is a step beyond what the homework asked for.
 - **element.after() DOM method** - Used this instead of appendChild/insertBefore for placing error messages. Cleaner API that we didn't use in the homework.
-

Outside sources

- **Bootstrap 5 documentation** (<https://getbootstrap.com/docs/5.0/>) - Used for the accordion component, navbar toggler, card layout, and grid system. Specifically the accordion page and navbar page.
- **MDN Web Docs** (<https://developer.mozilla.org/>) - Used for JavaScript references including element.after(), setTimeout, regex syntax for email validation, and event.preventDefault().
- **Homework assignments from CS 463** - Used as starting points. Specifically: homework 01 (HTML forms and images), homework 03 (CSS custom properties), homework 06 (Bootstrap cards and grid), and homework 07 (JavaScript DOM manipulation and event listeners).